1 Timeline

Classical computing was **deterministic** up until recently. More recently, we introduced **randomness** into algorithms.

In number theory, the `is_prime`$(n \in \mathbb{N})$ problem is of particular importance. For a long time, no known *efficient* (in terms of N, number of bits of n) algorithm was known. The trivial algorithm, searching for factors of n, runs in $\Theta\left(\sqrt{n}\right) = \Theta\left(\sqrt{2}^N\right)$ time.

Definition 1 (Efficient Algorithm, Polynomial-Time Algorithm)**.** We say an algorithm is **efficient** iff it runs in $\mathcal{O}\left(n^k\right)$ time for some $k \geqslant 0$, where n is the input size.

Soon, an efficient deterministic algorithm for `is_prime` was discovered, which runs in $\tilde{\mathcal{O}}\left(N^{12}\right)$ time (note: this was later refined to $\tilde{\mathcal{O}}\left(N^6\right)$). This algorithm is typically not used in practice: both because it is not particularly efficient, but also because there is a better alternative.

1.1 Randomized Algorithms

The *Miller-Rabin test* solves `is_prime` with high probability, and has a runtime of $\mathcal{O}\left(N^3\right)$. Without going into too much detail, one can argue that for composite $n \in \mathbb{N}$, at least $\frac{3}{4}$ of $\{2, ..., n-2\}$ is a "witness" to the fact that n is composite. By randomly sampling among $\{2, ..., n-2\}$, there is therefore $\geqslant 75\%$ chance of determining if n is composite. If n is prime, there is no such witness. By repeating this test k times, the probability of failing to detect that composite n is indeed composite is $\frac{1}{4^k}$. In essence, if the test detects a composite number, it is correct. If it detects a prime number (after k samples) the probability of failure is bounded by $\frac{1}{4^k}$. With large k, this failure can be made very rare.

Generally, all that a randomized algorithm needs is the concept of *sampling*. We assume that a single biased coin flip with a probability p of heads can be done in constant time. Consider an array A:

A_0	A_2	A_3	...	A_{n-1}

How can one randomly sample among the elements of this array, using coin flips? Assuming n is a power of two, each element of the array corresponds to a unique bit-string of length $\log_2 n = \log n$. Therefore, one can flip $\log n$ fair coins: the ith heads places a 1 in the ith digit of the bit string, and all other digits are left 0. The result is a random number (bit string) between 0 and $n-1$, each with probability $\mathbb{P}[i] = \left(\frac{1}{2}\right)^{\log n} = \frac{1}{n}$ (more info on this notation later).

What is the expected runtime of this randomization? Since we perform $\log n$ constant-time bit flips, it takes $\Theta(\log n)$ time to randomly sample from this array. For the sake of simplicity, we often abstract away this runtime and assume it can be sampled in constant time. **Importantly**, this means that we can only sample randomly from a polynomially-sized (as a function of n) space. If we fail to do so, the sampling time is no longer negligible.

2 Runtime bounds

Important note: the term *quantum advantage* is often referred to as *quantum supremacy* in class notes. Quantum advantage refers to a complexity improvement, quantum supremacy typically refers to quantum computers outperforming classical ones on any task at all. When saying quantum supremacy, we will mean quantum advantage.

When studying Quantum Computing, we often want to show *quantum supremacy* for a problem. In particular, it can be shown that some problems are $\Omega(f(n))$ on classic computers, but can be solved in $o(f(n))$ with quantum techniques. It is often easy (relatively speaking) to show that a problem is $O(f(n))$, simply by constructing an algorithm for the problem with number of queries $O(f(n))$. However, to show that a problem is $\Omega(f(n))$, one needs to show that for **all** algorithms for the problem, the query count is $\Omega(f(n))$.

The proof structure for this often follows: Construct an adversary $\text{Adv} : \text{Alg} \to \text{Inp}$ which maps algorithms to inputs for said algorithm. The adversary should be designed in such a way that given an algorithm, we construct an input which performs particularly poorly on the algorithm.

Example: Consider the `search`$(x \in A)$ problem. Again consider the array A from earlier, and assume it is not sorted. How can we find the index of a particular element x among the list? If the list were sorted, we could do so in $\mathcal{O}(\log n)$ queries, where $n = |A|$. Since it is not sorted, we must resort to linear search, which takes $\mathcal{O}(n)$ queries. Linear search functions by checking each index i for $0 \leqslant i < n$. This algorithm always correctly computes the `search` problem. Therefore, we know that `search` is $\mathcal{O}(n)$. But how can we show it is $\Omega(n)$?

We will now construct an adversary which generates trouble inputs for a general algorithm. Let Alg be an arbitrary algorithm for `search`. For simplicity, we will first assume that Alg is deterministic, and that Alg can only *query* the array A. There are no other tools available to it. Initialize $A = [1, ..., n]$ and $S = \varnothing$. Alg makes a series of "checks" on the array. That is, it checks the value $A[i]$ for some $i \in \{1, ..., n\}$. Lets look at what happens query by query. Suppose the algorithm queries index i during the kth query for $1 \leqslant k \leqslant n-1$. We split into two cases: if $A[i] = x$, then swap index i with some index $j \notin S \cup \{i\}$. Such a j exists since $|S \cup \{i\}| = k \leqslant n-1 < n$. If $A[i] \neq x$, then do nothing. At the end of this step, append i to S. Finally, restart Alg from scratch on the new array. In doing so, we can show inductively that for each iteration k, the algorithm takes at least $k-1$ iterations to find x.

At the end of iteration, we have a new array A such that Alg must make exactly $n-1$ queries before finding $A[i] = x$. This is our adversary: for any algorithm, we can construct an input where the search takes at least $n-1$ steps. In other words, no matter what algorithm we throw at the `search` problem, it *must* take at least $n-1$ queries. We have now shown that `search` is $\Omega(n)$.

In fact, I just **lied** to you. We made the assumption that the algorithm Alg is *deterministic*. That is, with the same information, the result and outcome of Alg is always the same. We showed that deterministic algorithms are $\Omega(n)$ on search. How do we know the same for probabilistic algorithms? The proof above does not work, since at any iteration i, rerunning the algorithm on the new array A may result in a different outcome in some iteration $j < i$. We will discuss later how we can prove such lower bounds when we take probabilistic algorithms into account.

2.1 Quantum Supremacy

Consider the `factor`(n) problem, which aims to factor integers n of the form $n = pq$ for p, q prime. No classical algorithm is known to solve such a problem in polynomial (in terms of $N = \log n$)

time. However, Shor's algorithm, a quantum algorithm, can do so in polynomial time $\mathcal{O}\left(N^3\right)$. Note however that the largest integer n factored via Shor's algorithm is $n = 21$.

Another famous example: consider the `search` problem again. As we have (attempted to) show, the `search` problem is $\Theta(n)$ classically. However, Grover's algorithm is a quantum algorithm for the `search` problem which computes in $\mathcal{O}(\sqrt{n})$ queries!

There is a class of problems called **NP-Complete**. Such problems are "easy to verify". That is, a solution can be checked in polynomial time.

More specifically, **NP-Complete** problems are a class of **NP** problems (problems which are easy to verify) which are all reducible to each other, or equivalently, reducible to `3-SAT` and vice versa (defined in a later lecture).

It is a famous problem to determine if P=NP. That is, if NP-Complete problems can be solved in polynomial time. In fact, quantum computers can run classical problems with polynomial overhead. Therefore, if P=NP then quantum computers can also run NP-Complete problems in polynomial time. Contrapositively, if quantum computers are shown to be unable to run NP-Complete problems in polynomial time, then P $\neq$ NP.

3 Probability

We are given a **sample space** $S = \{\omega_i\}_{i\in\mathcal{F}}$ of **simple events** ω_i (indexed over some family $\mathcal{F}$). Given these, we can define a probability function:

Definition 2 (Probability)**.** Given a sample space S, a probability function $\mathbb{P} : 2^S \to [0,1]$ must satisfy the following axioms:

$$\forall \mathcal{E} \in 2^S,\ \mathbb{P}[\mathcal{E}] \geqslant 0 \qquad \text{(nonnegativity)}$$

$$\mathbb{P}[S] = 1 \qquad \text{(normalization)}$$

$$\forall \text{disjoint } \left\{A_i \in 2^S\right\}_{i\in\mathcal{F} \text{ countable}},\ \mathbb{P}\left[\bigcup_{i\in[n]} A_i\right] = \sum_{i\in[n]} \mathbb{P}[A_i] \qquad \text{(countable additivity)}$$

We call $\mathcal{E} \in 2^S$ **events**. For such events we define

$$\mathbb{P}[\mathcal{E}] = \sum_{\omega\in\mathcal{E}} \mathbb{P}[\omega]$$

Definition 3 (Random Variable)**.** Any function $X : S \to \mathbb{R}$ is called a random variable.

For further notation, we define

$$\begin{aligned}\mathbb{P}[X = r] &= \sum_{X(\omega)=r} \mathbb{P}[\omega] \\ &= \mathbb{P}[\{\omega \in S : X(\omega) = r\}] \\ &= \mathbb{P}\left[X^{-1}(r)\right]\end{aligned}$$

Example: Randomized algorithms for search. Consider a randomized algorithm for `search` as follows: randomly sample from elements of the array A exactly k times. What is the probability after a single sample we arrive at x? Let's define X_1 the index we randomly choose uniformly. Then, $\mathbb{P}[X_1 = i_x] = \frac{1}{n}$ (where i_x is the index where x is located in A). One can show (via independence, not yet defined) that the probability of not finding the element x after k iterations is

$$\begin{aligned}\mathbb{P}[X_1 \neq i_x \cap ... \cap X_k \neq i_x] &= \prod_{j \in [k]} \mathbb{P}[X_j \neq i_x] \\ &= \prod_{j \in [k]} \left(1 - \frac{1}{n}\right) \\ &= \left(1 - \frac{1}{n}\right)^k\end{aligned}$$

And thus, the probability of finding the index at some point in the first k iterations is $1 - \left(1 - \frac{1}{n}\right)^k$.

Next time, we will go over expected value. If we define X the number of checks until we find the correct index, we can show that $\mathbb{E}[X] = n$. That is, we expect n lookups before we find the correct index.

Bibliography